Understanding a 2D Finite Element Solver

A line-by-line explanation of `fem2d.cpp`
Constant-strain triangles, plane stress, C++ and the Eigen library

Written for a reader who is new to the Finite Element Method (FEM) and new to Eigen.
Goal: after reading this you can explain every step of the code to a professor.

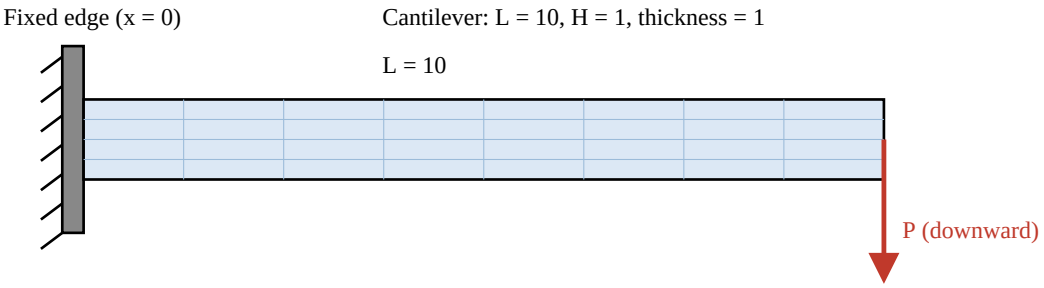
0. The 60-second summary (say this first)

Problem. A thin steel cantilever beam is clamped at its left edge and pushed down at its right edge. How much does the tip move, and how are stresses distributed inside it?

Idea. The beam is a continuum with infinitely many points, so we cannot solve for the motion of every point. FEM cuts the beam into many small triangles (*elements*) joined at corner points (*nodes*). We only solve for the displacement of the nodes. Inside each triangle the displacement is assumed to vary linearly, which is accurate when triangles are small.

Method. Each triangle behaves like a tiny spring with a 6 x 6 stiffness matrix *Ke*. Adding all the small matrices into one big matrix *K* gives a single linear system **K u = f** (stiffness times displacement equals force), exactly like a spring law F = k x but with thousands of unknowns. We apply the boundary conditions, solve for *u*, then compute strain and stress from *u*.

Check. We compare the computed tip deflection with the textbook beam formula P L^3 / (3 E I). Our value is about 17% smaller in magnitude, which is expected for this simple element (Section 3.9 explains why).



The program pipeline, with the code lines that implement each step:

Step	What happens	Code lines
1	Choose geometry, material, loads	67-70
2	Build the mesh (nodes + triangles)	18-37, 72
3	Build the material matrix D	39-47, 74
4	For each triangle: compute B, then Ke, and add it into the global K	49-64, 76-87
5	Build the load vector f	89-94
6	Apply boundary conditions (remove fixed DOFs)	96-115
7	Solve K u = f	117-123
8	Check against beam theory, post-process stress, write CSV files	125-149

1. The physics you need

1.1 Displacement and strain

When a body is loaded, every point moves. In 2D, the point at (x, y) moves by *u(x,y)* in the x direction and *v(x,y)* in the y direction. Together these are the **displacement field**.

Strain measures how much the material is stretched or sheared, not how far it moved. Moving the whole body by 5 mm is not a deformation; stretching it is. In 2D there are three strain components:

strain_xx = du/dx	stretch along x
strain_yy = dv/dy	stretch along y
gamma_xy = du/dy + dv/dx	shear (change of angle)

We pack them in a 3-component vector **eps = [strain_xx, strain_yy, gamma_xy]**. Strain has no units (it is a ratio of lengths).

1.2 Stress and Hooke's law

Stress is internal force per unit area (units: N/mm^2 = MPa). For a simple bar, Hooke's law says stress = E x strain, where *E* is Young's modulus (stiffness of the material; about 200,000 MPa for steel). In 2D the same idea is written with a matrix: **sigma = D eps**, where sigma = [sigma_xx, sigma_yy, tau_xy] and **D** is the 3 x 3 *constitutive matrix* that holds the material properties.

1.3 Plane stress and the D matrix

A real body is 3D. We simplify to 2D with the **plane stress** assumption: the body is a thin plate loaded only in its own plane, so the stress through the thickness is zero (sigma_zz = 0). For a plate of thickness t this gives:

1	nu	0
nu	1	0
0	0	(1 - nu)/2

multiplied by the scalar **E / (1 - nu^2)**, where *nu* (Poisson's ratio, about 0.3 for steel) says how much the material shrinks sideways when stretched. The bottom-right entry equals the shear modulus G = E / (2(1 + nu)), which you can verify by algebra. This is exactly what `planeStressD` builds in the code.

Plane strain is the other common 2D choice (a very thick body, like a dam cross-section). Only the D matrix changes; everything else in the code stays the same.

1.4 Why "K u = f"? (energy view)

The stored strain energy in a body is U = (1/2) times the integral over the volume of (eps^T D eps). The work done by the loads is W = u^T f. Nature chooses the displacement that minimizes the **total potential energy** Pi = U - W. After discretization U becomes (1/2) u^T K u, and setting the derivative of Pi with respect to u to zero gives

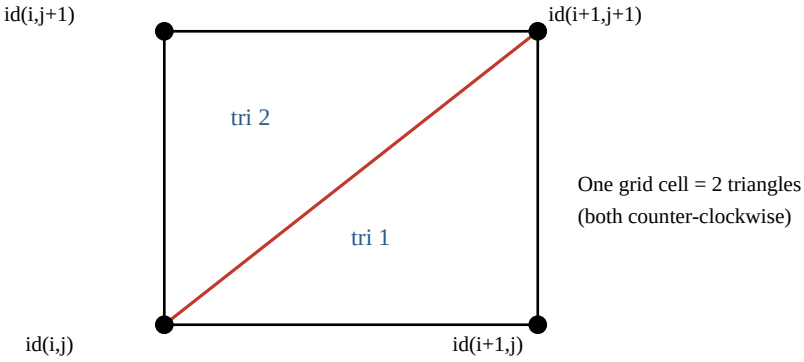
K u = f

K is the **global stiffness matrix**. Entry K(a,b) answers: "how much force is needed at degree of freedom a to hold a unit displacement at degree of freedom b, with all other displacements zero?" K is symmetric. It becomes invertible only after we constrain the body so it cannot move as a rigid object.

2. The finite element idea, step by step

2.1 Mesh, nodes, elements, degrees of freedom

A **mesh** is a set of **nodes** (points with coordinates) and **elements** (triangles listing three node indices). Each node can move in x and y, so each node has **2 degrees of freedom (DOF)**. In this code node number n owns DOF **2n** (x motion) and **2n+1** (y motion). For our grid (40 x 4 cells): nodes = 41 x 5 = 205, DOF = 410, triangles = 2 x 40 x 4 = 320. So K is 410 x 410.



2.2 Shape functions: interpolating inside a triangle

We only know displacement at the three corners: (u1,v1), (u2,v2), (u3,v3). Inside the triangle we *assume* it varies linearly:

u(x,y) = N1 u1 + N2 u2 + N3 u3, and the same for v.

The **shape functions** N1, N2, N3 are linear functions with N_i = 1 at node i and 0 at the other two nodes, and N1+N2+N3 = 1 everywhere. Writing N_i = (a_i + b_i x + c_i y) / (2A) where A is the triangle area, the useful coefficients are:

b1 = y2 - y3	b2 = y3 - y1	b3 = y1 - y2
c1 = x3 - x2	c2 = x1 - x3	c3 = x2 - x1

and the area is A = (1/2)(x1 b1 + x2 b2 + x3 b3). These are exactly the arrays `b[3]`, `c[3]` and the `area` line in `bMatrix`. **A is positive only if the nodes are listed counter-clockwise**; that is why the mesh generator orders them carefully.

2.3 The B matrix: strain from nodal displacements

Because the shape functions are linear, their derivatives are constants (b_i/2A and c_i/2A). Differentiating u and v gives strain directly from the six nodal values u_e = [u1, v1, u2, v2, u3, v3]:

eps = B u_e, with

b1	0	b2	0	b3	0
0	c1	0	c2	0	c3
c1	b1	c2	b2	c3	b3

all divided by 2A. The strain is the **same everywhere inside the triangle**, which is why this element is called the **Constant Strain Triangle (CST)**. Row 1 gives strain_xx, row 2 gives strain_yy, row 3 gives the shear strain.

2.4 Element stiffness matrix

Plug eps = B u_e into the energy U_e = (1/2) times the integral of (eps^T D eps) over the element. Since B and D are constant, the integral is just (value) x (volume = t A):

Ke = t A B^T D B (a 6 x 6 matrix)

Check the sizes: B^T is 6x3, D is 3x3, B is 3x6, so the product is 6x6. Ke(a,b) is the force at the element's local DOF a when local DOF b is displaced by one unit.

2.5 Assembly: building the global K

Each element has three nodes; the 6 local DOFs map to 6 global DOFs through the array `dof[6] = {2*n1, 2*n1+1, 2*n2, 2*n2+1, 2*n3, 2*n3+1}`. Entry Ke(a,b) is **added** to K(dof[a], dof[b]). When several triangles share a node, their contributions simply sum, which represents the physical fact that the node is pulled by every element around it. This "scatter and add" step is called **assembly**.

In the code the contributions are stored as *triplets* (row, column, value). Duplicate (row, column) pairs are summed automatically when the sparse matrix is built, so assembly happens inside `setFromTriplets`.

2.6 Loads and boundary conditions

The load vector f has one entry per DOF. We spread the total load P equally over the nodes of the right edge (entry 2n+1 is the y direction). This is a simplification; a rigorous approach gives the two corner nodes half the share of interior ones.

If we try to solve K u = f right away, K is **singular**: the whole beam could slide or rotate freely without any strain, so the solution is not unique. We must constrain the body. The fixed (left) edge nodes get u = v = 0. The method used here is **elimination**: remove the rows and columns of constrained DOFs, leaving a smaller system **K_ff u_f = f_f** for the free DOFs only. The constrained displacements are zero, so nothing is lost.

2.7 Solving

K is large but **sparse** (mostly zeros), because a DOF only interacts with DOFs on neighboring triangles. We store only the nonzeros and use a sparse direct solver. K_ff is symmetric positive definite, so the **LDL^T factorization** (a variant of Cholesky) is the right tool: it splits the matrix into a lower triangular L and a diagonal D so that solving becomes two cheap triangular solves.

2.8 Post-processing: stress and von Mises

With u known, collect each triangle's six nodal values u_e, compute strain eps = B u_e, then stress sigma = D eps. To get one number per element, we use the **von Mises equivalent stress**, the standard yield indicator for metals. For plane stress:

sigma_vm = sqrt(sigma_xx^2 - sigma_xx sigma_yy + sigma_yy^2 + 3 tau_xy^2)

2.9 Verification and why the answer is 17% off

For a slender cantilever with a tip load P, Euler-Bernoulli beam theory gives the tip deflection **delta = P L^3 / (3 E I)** with second moment of area **I = t H^3 / 12**. With P = -10, L = 10, E = 200000, I = 1/12 this gives -0.2000. Our solver gives -0.1654.

The reason: bending needs strain that varies *linearly* through the beam height (tension on top, compression at the bottom), but each CST can only represent a *constant* strain. The element is therefore artificially stiff in bending, a phenomenon called **shear locking**. It improves with a finer mesh (try ny = 8, 16) or with better elements (6-node triangles, 4-node quadrilaterals). Beam theory itself is also only an approximation for short, thick beams.

Good presentation point: a verification step (compare with a known analytical result) is what makes a numerical code trustworthy. Showing the convergence as the mesh is refined is the standard way to demonstrate it.

3. C++ and Eigen syntax primer

Everything used in the code is listed here with a plain explanation. Eigen is a header-only C++ library for linear algebra; install it with `sudo apt install libeigen3-dev`, and the compile flag `-I /usr/include/eigen3` tells g++ where its headers live.

3.1 General C++ features used

Syntax	Meaning
`#include <vector>`	Copies a library header into your file so you can use its features.
`using Eigen::Matrix3d;`	Lets you write Matrix3d instead of Eigen::Matrix3d.
`typedef Eigen::Triplet<double> T;`	Gives a long type a short alias, T.
`struct Mesh { ... };`	A bundle of related variables (nodes + triangles) treated as one type.
`std::vector<X>`	A resizable array holding items of type X. v.size() is the length; v[i] reads item i (starting from 0).
`std::array<int,3>`	A fixed-size array of exactly 3 ints. Used for the 3 node indices of a triangle.
`push_back(x)` / `emplace_back(args)`	Append to a vector. emplace_back builds the item in place from constructor arguments.
`auto`	The compiler deduces the type for you.
`[&](int i,int j){ return ...; }`	A **lambda**: a small unnamed function defined inline. [&] lets it read the surrounding variables (here nx).
`for (auto& t : mesh.tris)`	Range-based loop: visits every triangle. The & means "refer to the original, do not copy".
`const Mesh& m`	Pass by **reference** (no copy) and read-only (const). Efficient for big objects.
`double& area`	A non-const reference parameter: the function writes the result into the caller's variable (an "output parameter").
`++i`, `i += 1`, `nf++`	Increment. nf++ returns the old value and then increments (used to number free DOFs 0,1,2,...).
`std::abs(x) < 1e-9`	Floating-point comparison with a tolerance, since 10.0*i/nx is rarely exactly equal to L.
`printf("%.6f", x)`	Formatted print. %f real number, %.6f six decimals, %d integer, \n newline.
`FILE* f = fopen(name,"w")`	Open a file for writing. fprintf writes formatted text into it, fclose closes it.
`std::vector<bool> fixed`	A list of true/false flags, one per DOF (true = constrained).

3.2 Eigen dense types

Syntax	Meaning
`Matrix<double,3,6> B;`	A fixed-size matrix: 3 rows, 6 columns, entries of type double. Sizes known at compile time, so it is fast.
`Matrix3d`, `Vector2d`, `Vector3d`	Shortcuts: Matrix3d = Matrix<double,3,3>, Vector2d = 2 doubles, etc.
`VectorXd`	A vector whose length is chosen at run time (the X means "dynamic").
`VectorXd::Zero(n)`	Creates a length-n vector filled with zeros.
`B.setZero();`	Fills an existing matrix with zeros. Always do this before filling selected entries.
`B(i,j)`, `u(i)`	Access entry in row i, column j (indices start at 0). Round brackets, not square.
`D << 1, nu, 0, nu, 1, 0, 0, 0, c;`	Comma initializer: fills the matrix row by row.
`p.x()`, `p.y()`	First and second components of a Vector2d.
`B.transpose()`	Returns the transpose.
`A * B`	True matrix multiplication (not element-wise). Dimensions must match: (m x k)(k x n) gives (m x n).
`c * D`, `B / s`	Multiply or divide every entry by a scalar.

3.3 Eigen sparse types and the solver

Syntax	Meaning
`Eigen::Triplet<double>(r,c,v)`	One nonzero entry: row r, column c, value v. We collect many of these in a std::vector.
`SparseMatrix<double> K(n,n);`	An n x n matrix that stores only nonzero entries (column-major order internally).
`K.setFromTriplets(b, e)`	Builds the matrix from the list of triplets between iterators b and e. **Duplicate positions are summed**, which is precisely what assembly needs.
`K.outerSize()`	Number of columns (for a column-major matrix).
`SparseMatrix<double>::InnerIterator it(K,k)`	Walks over the nonzeros of column k. it.row(), it.col(), it.value() give the position and value; `++it` moves on; the loop stops when `it` is false.
`SimplicialLDLT<SparseMatrix<double>> solver(Kff);`	Constructing it immediately computes the LDL^T factorization of the symmetric matrix Kff.
`solver.info() != Eigen::Success`	Error check: did the factorization work? A singular (unconstrained) matrix would fail here.
`solver.solve(ff)`	Solves Kff x = ff and returns x.

Why sparse? A dense 410 x 410 matrix holds 168,100 numbers, and a real 3D model with a million DOFs would need about 8 terabytes dense. In FEM each row has only a handful of nonzeros, so sparse storage and sparse solvers are essential for scaling up. This is also the entry point to HPC topics such as iterative solvers and parallelism.

4. Line-by-line walkthrough

Line numbers match the numbers printed beside the code in `fem2d.cpp`.

4.1 Header and setup (lines 1-16)

```
 1  // Minimal 2D FEM (constant-strain triangles, plane stress, linear elastic)
 2  // Build: g++ -O2 -std=c++17 -I /usr/include/eigen3 fem2d.cpp -o fem2d
 3  // Test: cantilever, left edge fixed, downward point-load distributed on right edge.
 4
 5  #include <Eigen/Dense>
 6  #include <Eigen/Sparse>
 7  #include <array>
 8  #include <cmath>
 9  #include <cstdio>
10  #include <vector>
11
12  using Eigen::Matrix3d;
13  using Eigen::Matrix;
14  using Eigen::Vector2d;
15  using Eigen::VectorXd;
16  typedef Eigen::Triplet<double> T;
```

- Lines 1-3 are comments describing the program and how to build it.
- Lines 5-6 include Eigen: **Dense** gives small/medium matrices and vectors, **Sparse** gives sparse matrices and the sparse solvers.
- Lines 7-10 include standard C++ pieces: fixed arrays, math (sqrt, abs), C-style printing/file I/O, and std::vector.
- Lines 12-15 import the Eigen names we use so we do not need to type the long prefix.
- Line 16 shortens Eigen::Triplet<double> to **T**. A triplet is one entry (row, column, value) of a sparse matrix.

4.2 The mesh data structure (lines 18-21)

```
18  struct Mesh {
19      std::vector<Vector2d> nodes;
20      std::vector<std::array<int, 3>> tris;
21  };
```

- **nodes**: list of 2D points. nodes[7] is the (x,y) of node number 7.
- **tris**: list of triangles. Each triangle stores three *node numbers* (not coordinates). This indirection is how elements share nodes.

4.3 Generating the grid (lines 23-37)

```
23  // Structured nx-by-ny grid on [0,L]x[0,H], each cell split into 2 triangles.
24  Mesh makeGrid(double L, double H, int nx, int ny) {
25      Mesh m;
26      for (int j = 0; j <= ny; ++j)
27          for (int i = 0; i <= nx; ++i)
28              m.nodes.emplace_back(L * i / nx, H * j / ny);
29      auto id = [&](int i, int j) { return j * (nx + 1) + i; };
30      for (int j = 0; j < ny; ++j)
31          for (int i = 0; i < nx; ++i) {
32              // counter-clockwise ordering matters (positive area)
33              m.tris.push_back({id(i, j), id(i + 1, j), id(i + 1, j + 1)});
34              m.tris.push_back({id(i, j), id(i + 1, j + 1), id(i, j + 1)});
35          }
36      return m;
37  }
```

- Lines 26-28: loop over rows j and columns i of grid points and create a node at (L i/nx, H j/ny). Node numbering runs left to right, then bottom to top.

- Line 29: the lambda **id(i,j) = j*(nx+1) + i** converts a grid position into a node number.
- Lines 30-35: for every rectangular cell create **two** triangles by cutting along the diagonal, using the four corner node numbers (see the figure in Section 2.1).
- Both triangles list their nodes counter-clockwise; otherwise the area computed later would be negative.
- Line 36 returns the finished mesh.

4.4 The material matrix D (lines 39-47)

```
39  // Plane-stress constitutive matrix D
40  Matrix3d planeStressD(double E, double nu) {
41      Matrix3d D;
42      double c = E / (1.0 - nu * nu);
43      D << 1, nu, 0,
44           nu, 1, 0,
45           0, 0, (1 - nu) / 2;
46      return c * D;
47  }
```

- Takes Young's modulus E and Poisson's ratio nu and returns the 3 x 3 plane-stress matrix from Section 1.3.
- Line 42 computes the common factor E/(1 - nu^2).
- Lines 43-45 fill the matrix with the comma initializer, row by row. Line 46 multiplies by the factor.

4.5 The B matrix and the area (lines 49-64)

```
49  // B matrix (3x6) and area for one triangle
50  Matrix<double, 3, 6> bMatrix(const Mesh& m, const std::array<int, 3>& t, double& area) {
51      const Vector2d &p1 = m.nodes[t[0]], &p2 = m.nodes[t[1]], &p3 = m.nodes[t[2]];
52      double b[3] = {p2.y() - p3.y(), p3.y() - p1.y(), p1.y() - p2.y()};
53      double c[3] = {p3.x() - p2.x(), p1.x() - p3.x(), p2.x() - p1.x()};
54      area = 0.5 * (p1.x() * b[0] + p2.x() * b[1] + p3.x() * b[2]);
55      Matrix<double, 3, 6> B;
56      B.setZero();
57      for (int k = 0; k < 3; ++k) {
58          B(0, 2 * k) = b[k];
59          B(1, 2 * k + 1) = c[k];
60          B(2, 2 * k) = c[k];
61          B(2, 2 * k + 1) = b[k];
62      }
63      return B / (2.0 * area);
64  }
```

- Inputs: the mesh, the triangle (three node numbers) and a reference **area** which the function fills in for the caller.
- Line 51: grabs the three corner points p1, p2, p3 as references (no copying).
- Lines 52-53: the coefficients b_i and c_i from Section 2.2.
- Line 54: the triangle area A = (1/2)(x1 b1 + x2 b2 + x3 b3).
- Lines 55-56: create B (3 rows, 6 columns) and zero it.
- Lines 57-62: for each node k place b_k and c_k in the right spots. Columns 2k and 2k+1 correspond to the x and y motion of node k. Row 0 is strain_xx, row 1 is strain_yy, row 2 is shear.
- Line 63: divide everything by 2A and return.

4.6 Parameters and setup in main (lines 66-74)

```
66  int main() {
67      const double L = 10.0, H = 1.0, thick = 1.0;
68      const double E = 200e3, nu = 0.3;  // MPa-ish units, keep consistent!
69      const int nx = 40, ny = 4;
70      const double P = -10.0;            // total downward load on right edge
71
72      Mesh mesh = makeGrid(L, H, nx, ny);
73      const int N = (int)mesh.nodes.size(), ndof = 2 * N;
74      Matrix3d D = planeStressD(E, nu);
```

- Geometry: length 10, height 1, thickness 1. Material: E = 200,000 and nu = 0.3 (steel in MPa and mm). **Units must be consistent**: here N, mm, MPa.
- nx = 40 by ny = 4 cells. P = -10 is the total downward force (negative y).
- Line 72: builds the mesh. Line 73: N = number of nodes, ndof = 2N total degrees of freedom. Line 74: builds D once, since it is the same for every element.

4.7 Assembling the global stiffness matrix (lines 76-87)

```
76      // --- Assemble global stiffness ---
77      std::vector<T> trip;
78      for (auto& t : mesh.tris) {
79          double A;
80          auto B = bMatrix(mesh, t, A);
81          Matrix<double, 6, 6> Ke = thick * A * B.transpose() * D * B;
82          int dof[6] = {2 * t[0], 2 * t[0] + 1, 2 * t[1], 2 * t[1] + 1, 2 * t[2], 2 * t[2] + 1};
83          for (int a = 0; a < 6; ++a)
84              for (int b = 0; b < 6; ++b) trip.emplace_back(dof[a], dof[b], Ke(a, b));
85      }
86      Eigen::SparseMatrix<double> K(ndof, ndof);
87      K.setFromTriplets(trip.begin(), trip.end());
```

- Line 77: an empty list of triplets that will collect every contribution.
- Lines 78-85: loop over every triangle t. Compute its B and area (line 80), then the element stiffness **Ke = thickness x A x B^T D B** (line 81).
- Line 82: **dof[6]** maps the element's six local DOFs to global DOF numbers.
- Lines 83-84: double loop over the 6 x 6 entries; each becomes a triplet (global row, global column, value). This is the scatter step of assembly.
- Lines 86-87: create the empty 410 x 410 sparse matrix K and fill it from the triplets. Duplicates (from shared nodes) are summed here, completing the assembly.

4.8 Building the load vector (lines 89-94)

```
89      // --- Loads: spread P over right-edge nodes ---
90      VectorXd f = VectorXd::Zero(ndof);
91      std::vector<int> right;
92      for (int i = 0; i < N; ++i)
93          if (std::abs(mesh.nodes[i].x() - L) < 1e-9) right.push_back(i);
94      for (int n : right) f(2 * n + 1) = P / right.size();  // crude but fine for a first test
```

- Line 90: f = vector of zeros, one entry per DOF.
- Lines 91-93: find all nodes on the right edge (x equals L within a tiny tolerance).
- Line 94: for each of them, put P divided by the number of right-edge nodes into the y-component (index 2n+1).

4.9 Boundary conditions by elimination (lines 96-115)

```
 96      // --- Boundary conditions: fix left edge (eliminate rows/cols) ---
 97      std::vector<bool> fixed(ndof, false);
 98      for (int i = 0; i < N; ++i)
 99          if (std::abs(mesh.nodes[i].x()) < 1e-9) fixed[2 * i] = fixed[2 * i + 1] = true;
100
101      std::vector<int> freeMap(ndof, -1);
102      int nf = 0;
103      for (int d = 0; d < ndof; ++d)
104          if (!fixed[d]) freeMap[d] = nf++;
105
106      std::vector<T> tf;
107      for (int k = 0; k < K.outerSize(); ++k)
108          for (Eigen::SparseMatrix<double>::InnerIterator it(K, k); it; ++it)
109              if (freeMap[it.row()] >= 0 && freeMap[it.col()] >= 0)
110                  tf.emplace_back(freeMap[it.row()], freeMap[it.col()], it.value());
111      Eigen::SparseMatrix<double> Kff(nf, nf);
112      Kff.setFromTriplets(tf.begin(), tf.end());
113      VectorXd ff(nf);
114      for (int d = 0; d < ndof; ++d)
115          if (freeMap[d] >= 0) ff(freeMap[d]) = f(d);
```

- Lines 97-99: mark every DOF of the left-edge nodes as fixed (both x and y).
- Lines 101-104: **freeMap** renumbers the free DOFs 0,1,2,... and gives fixed DOFs the value -1. nf = number of free DOFs.
- Lines 106-112: walk through every nonzero of K; keep only those whose row *and* column are free, and store them with renumbered indices. This builds the reduced matrix **Kff**.
- Lines 113-115: build the reduced load vector ff by copying the free entries of f.

4.10 Solving and rebuilding the full vector (lines 117-123)

```
117      // --- Solve ---
118      Eigen::SimplicialLDLT<Eigen::SparseMatrix<double>> solver(Kff);
119      if (solver.info() != Eigen::Success) { std::puts("Factorization failed"); return 1; }
120      VectorXd uf = solver.solve(ff);
121      VectorXd u = VectorXd::Zero(ndof);
122      for (int d = 0; d < ndof; ++d)
123          if (freeMap[d] >= 0) u(d) = uf(freeMap[d]);
```

- Line 118: factorize Kff (LDL^T).
- Line 119: stop with a message if factorization fails (usually a missing or insufficient boundary condition).
- Line 120: solve for the free displacements uf.
- Lines 121-123: create the full displacement vector u (zeros), then put uf values back at their original DOF positions. Fixed DOFs stay at 0.

4.11 Verification against beam theory (lines 125-131)

```
125      // --- Sanity check against Euler-Bernoulli: delta = P L^3 / (3 E I) ---
126      double I = thick * H * H * H / 12.0;
127      double analytic = P * L * L * L / (3 * E * I);
128      double tip = 0;
129      for (int n : right) tip += u(2 * n + 1);
130      tip /= right.size();
131      std::printf("tip deflection FEM: %.6f   beam theory: %.6f\n", tip, analytic);
```

- $I = t\,H^3/12$ is the second moment of area of the rectangular cross-section; the analytic deflection is $P\,L^3/(3\,E\,I)$.
- Lines 128-130: average the y displacement of the right-edge nodes to get the FEM tip deflection.
- Line 131: prints both numbers so you can compare them.

4.12 Output and stress recovery (lines 133-149)

```
133     // --- Output for plotting / OpenGL: node positions + displacements, element von Mises ---
134     FILE* fn = std::fopen("nodes.csv", "w");
135     for (int i = 0; i < N; ++i)
136         std::fprintf(fn, "%f,%f,%f,%f\n", mesh.nodes[i].x(), mesh.nodes[i].y(), u(2 * i), u(2 * i + 1));
137     std::fclose(fn);
138
139     FILE* fe = std::fopen("elements.csv", "w");
140     for (auto& t : mesh.tris) {
141         double A;
142         auto B = bMatrix(mesh, t, A);
143         Matrix<double, 6, 1> ue;
144         for (int k = 0; k < 3; ++k) { ue(2 * k) = u(2 * t[k]); ue(2 * k + 1) = u(2 * t[k] + 1); }
145         Eigen::Vector3d s = D * B * ue;  // sxx, syy, txy
146         double vm = std::sqrt(s(0) * s(0) - s(0) * s(1) + s(1) * s(1) + 3 * s(2) * s(2));
147         std::fprintf(fe, "%d,%d,%d,%f\n", t[0], t[1], t[2], vm);
148     }
149     std::fclose(fe);
```

- Lines 134-137: write one line per node: x, y, u, v into **nodes.csv**. Add a scaled copy of (u,v) to (x,y) to draw the deformed shape.
- Lines 139-149: for every triangle, recompute B (line 142), gather its six nodal displacements u_e (lines 143-144), compute stress **s = D B u_e** (line 145), then the von Mises value (line 146), and write the three node indices plus von Mises into **elements.csv** (line 147).
- Remember: CST stress is constant per triangle, so the stress picture looks blocky; that is a limitation of the element, not a bug.

5. Questions your professor may ask

Q: Why do we need boundary conditions?

A: Without them the body can move as a rigid object, so K is singular and the displacement is not unique. Constraints remove the rigid-body modes.

Q: What does the stiffness matrix mean physically?

A: K(a,b) is the force at DOF a needed to produce a unit displacement at DOF b while all other DOFs are held at zero. K u = f is the generalization of F = k x.

Q: Why is the element called "constant strain"?

A: Shape functions are linear, so their derivatives, and therefore the strain B u_e, are constant inside each triangle.

Q: Why is your result 17% lower than beam theory?

A: CST cannot represent the linearly varying bending strain, so it is too stiff (shear locking). Refining the mesh or using higher-order elements reduces the error. Beam theory is itself an approximation.

Q: How do you know the code is correct?

A: We compare against an analytical solution, and we can check mesh convergence (error decreases as ny increases). Other checks: symmetry of K, equilibrium (reaction force equals applied load), and rigid-body motion producing zero strain.

Q: Why a sparse matrix and LDL^T?

A: Each DOF couples only to neighbors, so K has very few nonzeros. K_ff is symmetric positive definite, so LDL^T (Cholesky-type) is fast and stable. For millions of DOFs one moves to iterative solvers such as conjugate gradient with preconditioning.

Q: What is the difference between plane stress and plane strain?

A: Plane stress: thin plate, stress through the thickness is zero. Plane strain: very thick body, strain through the thickness is zero. Only the D matrix differs.

Q: How would you extend this?

A: 3D tetrahedra (B becomes 6 x 12, D 6 x 6), higher-order elements, dynamics (add a mass matrix and time integration), nonlinear materials (Newton iterations), and parallel solvers for HPC.

6. Known simplifications of this first version

- The load is spread equally over the right-edge nodes instead of using consistent nodal forces (end nodes should get half share).
- Only static linear analysis: small deformations and a linear elastic material.
- Stress is constant per triangle, so it is discontinuous between elements; smoothing at nodes is a common improvement.
- The mesh is a regular grid built in code; real models import meshes from tools such as Gmsh.

7. Mini glossary

Term	Meaning
Node	A mesh point where displacement is unknown.
Element	A small piece of the body (here a triangle) with 3 nodes.
DOF	Degree of freedom: an independent unknown, here the x or y displacement of a node.
Shape function	Function that interpolates the nodal values inside an element.
B matrix	Converts nodal displacements into strain.
D matrix	Converts strain into stress (material law).
Ke / K	Element stiffness matrix / global stiffness matrix.
Assembly	Adding element matrices into the global matrix at the right DOF positions.
Von Mises stress	A single equivalent stress used to judge yielding of metals.
Shear locking	Over-stiff behavior of low-order elements in bending.